Template Method Pattern

Justification

The design pattern chosen for the borrowing and loan processing workflow is the Template Method pattern.

Processing a loan in the Library Management System always follows the same sequence of steps: the user account is validated, the borrowing limit for that user type is checked, the availability of the requested book copy is confirmed, the loan-specific policy is applied, the loan record is created in the database, and a confirmation notification is sent. This skeleton is invariant. What varies from one user category to the next is how the borrowing limit is evaluated and how the loan policy is applied. A student has a limit of ten books and a loan duration of seven days. An academic staff member has a limit of twenty-five books and thirty days. A reference-only item has its own policy that restricts the loan to in-library use.

The Template Method pattern handles this by placing the invariant skeleton inside a single final method processLoan() in the abstract class LoanProcessor. This method calls the steps in the correct order and cannot be overridden. The two steps that differ between user types, validateBorrowingLimit() and applyLoanPolicy(), are declared as abstract hook methods. Each concrete subclass, StudentLoanProcessor, AcademicStaffLoanProcessor, and ReferenceOnlyLoanProcessor, provides its own implementation of those two hooks while inheriting all other steps unchanged.

This is the correct pattern to use rather than Strategy because the overall algorithm structure must be fixed. Strategy allows the caller to swap the entire algorithm. Here the caller must not be able to change the sequence of validation, availability check, record creation, and notification. Template Method enforces the sequence and delegates only the parts that legitimately differ.

The pattern is directly traceable to multiple requirements. BR_LMS_21 states that the system shall enforce loan duration policies based on user type and material type. BR_LMS_22 requires different borrowing limits per user category. BR_LMS_23 requires that returns automatically update item status, which fits inside the template alongside the loan lifecycle. BR_LMS_24 requires loan status tracking throughout the lifecycle, which the base class manages consistently across all subclasses.

The Single Responsibility Principle is satisfied because each subclass owns only the logic that is specific to its user type. The Open/Closed Principle is satisfied because adding a new user category such as a visiting researcher requires only adding a new subclass; the base class processLoan() template is not modified. The Liskov Substitution Principle holds because any concrete LoanProcessor can replace another in the system and the loan sequence will execute correctly.

UML Class Diagram